Darshan UNIVERSITY
योग: कर्मसु कौशलम्

Data Mining

Lab - 14

137 | Vishal Baraiya | 23010101014

Implement K- Means without Library

Sample data points

data = [[1, 2], [2, 3], [3, 4], [10, 11], [11, 12], [12, 13], [50, 51], [51, 52], [52, 53]]

In [1]:
```
import math
```

In [2]:
```
data = [
    [1, 2], [2, 3], [3, 4],
    [10, 11], [11, 12], [12, 13],
    [50, 51], [51, 52], [52, 53]
]
```

In [3]:
```
def distance(x1,x2):
    return math.sqrt(((x1[0] - x2[0])**2) + ((x1[1] - x2[1])**2))
```

In [4]:
```
distance([1,1],[1,1])
```

Out[4]:
```
0.0
```

In [5]:
```
def update_cluster_center(cluster_data):
    sum = [0,0]
    for i in cluster_data:
        sum[0] =  sum[0] + i[0]
```

```
        sum[1] =  sum[1] + i[1]
    return [sum[0]/len(cluster_data),sum[1]/len(cluster_data)]
```

In [6]:
```
update_cluster_center([[1,1],[2,2],[1,1]])
```

Out[6]:
```
[1.3333333333333333, 1.3333333333333333]
```

Now Implement code

In [7]:
```
import numpy as np
import random

def distance(a, b):
    # Example Euclidean distance for 2D points
    return np.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2)

def update_cluster_center(cluster_data, old_center=None):
    if len(cluster_data) == 0:
        # If cluster is empty, keep previous center or reinitialize randomly
        return old_center if old_center is not None else [random.uniform(0,10), ran
    sum_x = sum([i[0] for i in cluster_data])
    sum_y = sum([i[1] for i in cluster_data])
    return [sum_x / len(cluster_data), sum_y / len(cluster_data)]

def kmeans_du(k, data, iterations=5):
    # Select random initial centers
    center_data = [data[np.random.randint(0, len(data))] for _ in range(k)]
    print("Initial Centers:", center_data)

    for j in range(iterations):
        # Initialize empty clusters
        cluster_data = [[] for _ in range(k)]

        # Assign data points to nearest cluster
        for d in data:
            distances = [distance(c, d) for c in center_data]
            cluster_index = distances.index(min(distances))
            cluster_data[cluster_index].append(d)

        # Print cluster assignment
        print(f"\nIteration {j+1}")
        for i in range(k):
            print(f"Cluster {i}:", cluster_data[i])

        # Update cluster centers
        for i in range(k):
            center_data[i] = update_cluster_center(cluster_data[i], center_data[i])

        print("Updated Centers:", center_data)

    return center_data, cluster_data
```

In [8]:
```
kmeans_du(3,data)
```

```
Initial Centers: [[51, 52], [1, 2], [3, 4]]

Iteration 1
Cluster 0: [[50, 51], [51, 52], [52, 53]]
Cluster 1: [[1, 2], [2, 3]]
Cluster 2: [[3, 4], [10, 11], [11, 12], [12, 13]]
Updated Centers: [[51.0, 52.0], [1.5, 2.5], [9.0, 10.0]]

Iteration 2
Cluster 0: [[50, 51], [51, 52], [52, 53]]
Cluster 1: [[1, 2], [2, 3], [3, 4]]
Cluster 2: [[10, 11], [11, 12], [12, 13]]
Updated Centers: [[51.0, 52.0], [2.0, 3.0], [11.0, 12.0]]

Iteration 3
Cluster 0: [[50, 51], [51, 52], [52, 53]]
Cluster 1: [[1, 2], [2, 3], [3, 4]]
Cluster 2: [[10, 11], [11, 12], [12, 13]]
Updated Centers: [[51.0, 52.0], [2.0, 3.0], [11.0, 12.0]]

Iteration 4
Cluster 0: [[50, 51], [51, 52], [52, 53]]
Cluster 1: [[1, 2], [2, 3], [3, 4]]
Cluster 2: [[10, 11], [11, 12], [12, 13]]
Updated Centers: [[51.0, 52.0], [2.0, 3.0], [11.0, 12.0]]

Iteration 5
Cluster 0: [[50, 51], [51, 52], [52, 53]]
Cluster 1: [[1, 2], [2, 3], [3, 4]]
Cluster 2: [[10, 11], [11, 12], [12, 13]]
Updated Centers: [[51.0, 52.0], [2.0, 3.0], [11.0, 12.0]]
```

Out[8]:
```
([[51.0, 52.0], [2.0, 3.0], [11.0, 12.0]],
 [[[50, 51], [51, 52], [52, 53]],
  [[1, 2], [2, 3], [3, 4]],
  [[10, 11], [11, 12], [12, 13]]])
```

Implement K-Medoids without Library

Sample data points

data = [[1, 2], [2, 3], [3, 4], [10, 11], [11, 12], [12, 13], [50, 51], [51, 52], [52, 53]]

In [9]:
```
import random
import math
```

In [10]:
```
def euclidean_distance(p1, p2):
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(p1, p2)))
```

In [11]:
```
def assign_points(data, medoids):
    clusters = {i: [] for i in range(len(medoids))}
    for point in data:
```

```
        distances = [euclidean_distance(point, medoid) for medoid in medoids]
        nearest = distances.index(min(distances))
        clusters[nearest].append(point)
    return clusters
```

In [12]:
```
def calculate_cost(clusters, medoids):
    cost = 0
    for i, points in clusters.items():
        for p in points:
            cost += euclidean_distance(p, medoids[i])
    return cost
```

In [13]:
```
def k_medoids(data, k, max_iter=100):
    # Step 1: Randomly select initial medoids
    medoids = random.sample(data, k)

    for _ in range(max_iter):
        clusters = assign_points(data, medoids)
        current_cost = calculate_cost(clusters, medoids)

        best_medoids = medoids[:]
        improved = False

        # Step 2: Try swapping medoids with non-medoids
        for i in range(len(medoids)):
            for candidate in data:
                if candidate not in medoids:
                    new_medoids = medoids[:]
                    new_medoids[i] = candidate
                    new_clusters = assign_points(data, new_medoids)
                    new_cost = calculate_cost(new_clusters, new_medoids)

                    if new_cost < current_cost:
                        best_medoids = new_medoids
                        current_cost = new_cost
                        improved = True

        medoids = best_medoids
        if not improved:
            break  # convergence

    final_clusters = assign_points(data, medoids)
    return medoids, final_clusters
```

In [14]:
```
k = 3
medoids, clusters = k_medoids(data, k)
```

In [15]:
```
print("Final Medoids:", medoids)
print("Clusters:")
for i, points in clusters.items():
    print(f"Cluster {i+1}: {points}")
```

```
Final Medoids: [[51, 52], [11, 12], [2, 3]]
Clusters:
Cluster 1: [[50, 51], [51, 52], [52, 53]]
Cluster 2: [[10, 11], [11, 12], [12, 13]]
Cluster 3: [[1, 2], [2, 3], [3, 4]]
```

In [ ]: